

Exercícios — Aula 11 — Mergesort

Técnicas de Programação - 2026/02

Última atualização: October 5, 2026

Instruções

- Antes de pensar no mergesort completo, domine a intercalação de duas partes ordenadas.
- Use intervalos abertos no fim: `[inicio, meio)` e `[meio, fim)`.
- Em simulações de merge, acompanhe `i`, `j`, `k`, o valor copiado e o vetor auxiliar.

Exercício 1 — Simulação de MERGE

Simule o algoritmo para:

- `A1 = [4, 4, 7, 8]`;
- `A2 = [1, 3, 5, 6]`.

Algorithm 1: MergeDoisArrays

Result: Executa `MERGE-DOIS-ARRAYS`.

Input : arrays ordenados `A1` e `A2`

Output: array ordenado `AUX`

`AUX` $\leftarrow$ `NOVO-ARRAY`(`TAMANHO`(`A1`) + `TAMANHO`(`A2`))

`i` $\leftarrow$ 0

`j` $\leftarrow$ 0

`k` $\leftarrow$ 0

while `i` < `TAMANHO`(`A1`) **and** `j` < `TAMANHO`(`A2`) **do**

if `A1`[`i`] $\leq$ `A2`[`j`] **then**

`AUX`[`k`] $\leftarrow$ `A1`[`i`]

`i` $\leftarrow$ `i` + 1

end

else

`AUX`[`k`] $\leftarrow$ `A2`[`j`]

`j` $\leftarrow$ `j` + 1

end

`k` $\leftarrow$ `k` + 1

end

while `i` < `TAMANHO`(`A1`) **do**

`AUX`[`k`] $\leftarrow$ `A1`[`i`]

`i` $\leftarrow$ `i` + 1

`k` $\leftarrow$ `k` + 1

end

while `j` < `TAMANHO`(`A2`) **do**

`AUX`[`k`] $\leftarrow$ `A2`[`j`]

`j` $\leftarrow$ `j` + 1

`k` $\leftarrow$ `k` + 1

end

return `AUX`

k	i antes	j antes	comparação	valor copiado	AUX parcial
0					
1					
2					
3					
4					
5					
6					
7					

Qual é o valor final de AUX?

Exercício 2 — Divisões do mergesort

Para $v = [8, 3, 7, 1, 5, 2, 6, 4]$, desenhe a árvore de divisões do mergesort até chegar a intervalos de tamanho 1.

Use a notação $[\text{inicio}, \text{fim})$.

```
[0, 8)
|-- [0, 4)
|   |-- ...
|   `-- ...
`-- [4, 8)
     |-- ...
     `-- ...
```

Depois, marque em que ordem os merges acontecem na volta da recursão.

Exercício 3 — Completar pseudocódigo de merge em um array

Complete o pseudocódigo para intercalar duas metades ordenadas de v .

Algorithm 2: Merge

Result: Executa MERGE.

Input : array v , array aux , int $inicio$, int $meio$, int fim

Output: $v[inicio..fim]$ ordenado

$i \leftarrow inicio$

$j \leftarrow meio$

$k \leftarrow inicio$

while $i < meio$ **and** $j < fim$ **do**

if _____ **then**

$aux[k] \leftarrow v[i]$

$i \leftarrow i + 1$

end

else

$aux[k] \leftarrow v[j]$

$j \leftarrow j + 1$

end

$k \leftarrow k + 1$

end

while _____ **do**

$aux[k] \leftarrow v[i]$

$i \leftarrow i + 1$

$k \leftarrow k + 1$

end

while _____ **do**

$aux[k] \leftarrow v[j]$

$j \leftarrow j + 1$

$k \leftarrow k + 1$

end

for $p \leftarrow inicio$ **to** $fim - 1$ **do**

end

Teste com $v = [2, 7, 9, 1, 5, 8]$, $inicio = 0$, $meio = 3$, $fim = 6$.

Exercício 4 — Estabilidade no merge

Considere:

- esquerda: [(4, "A"), (4, "B"), (7, "C")];
- direita: [(4, "D"), (5, "E")].

O merge usa a condição `esquerda[i].chave <= direita[j].chave`.

1. Simule a intercalação.
2. Em que ordem aparecem "A", "B" e "D"?
3. O que mudaria se a condição fosse `<` em vez de `<=`?
4. Explique por que essa escolha está ligada à estabilidade.

Exercício 5 — Níveis e custo $n \log n$

Complete a tabela considerando arrays cujo tamanho é potência de 2.

tamanho n	quantidade aproximada de níveis	trabalho total por nível	trabalho total aproximado
2			
4			
8			
16			
32			

Explique por que cada nível faz trabalho linear e por que há cerca de $\log_2(n)$ níveis.

Exercício 6 — Mergesort completo

Escreva pseudocódigo para MERGESORT.

Contrato:

- entrada: array v ;
- saída: v ordenado;

Depois, teste manualmente com:

- $v = []$;
- $v = [5]$;
- $v = [2, 1]$;
- $v = [4, 1, 3, 2]$.